

Microsoft AI Agent Stack – Complete Training Reference (Foundry)

This page is a **training-grade, end-to-end reference** for understanding how **Foundry OpenAI SDK**, **Microsoft Foundry Agent Service**, **Hosted runtime concepts**, and **Microsoft Agent Framework (MAF)** fit together architecturally, when to use each SDK, and how to combine them correctly in real systems.

Terminology correction (exam-critical): In Foundry Agent Service, “*hosted*” does *NOT* mean *prompt-only*, *configuration-only*, or *no-code*. It refers to **where the agent runtime executes**.

1. The Building Blocks (Correct Foundry Mental Model)

In a Foundry-only architecture, the agent stack consists of **four orthogonal concerns**:

Layer	What it is	Primary responsibility
Foundry OpenAI SDK	Model access layer	LLMs, embeddings, multimodal inference
Foundry Agent Service	Managed runtime & ops plane	Hosting, identity, memory, tools, governance
Hosted runtime	Deployment model	Where agent logic executes (Foundry vs your compute)
Microsoft Agent Framework (MAF)	Agent SDK	Explicit logic, workflows, multi-agent systems

Key idea: *Hosted* describes **runtime placement**, not agent sophistication.

2. Foundry OpenAI SDK (Model Layer)

The **Foundry OpenAI SDK** is the *only* supported model interface in this architecture.

What it provides

- Access to GPT-class models, reasoning models, embeddings, multimodal models
- Enterprise guardrails, logging, quotas
- Identity-bound usage via Foundry projects

What it does not provide

- No agents
- No tool execution

- No memory
- No orchestration

Correct usage

Use Foundry OpenAI SDK **inside**:

- Application code (services, batch jobs, APIs)
- MAF agents and workflows
- Tools invoked by agents

Python example – direct Foundry model call (runnable)

```

1  import os
2  from dotenv import load_dotenv
3
4  from azure.ai.projects import AIProjectClient
5  from azure.identity import DefaultAzureCredential
6
7  load_dotenv()
8
9  endpoint = os.environ["FOUNDRY_PROJECT_ENDPOINT"]
10 model = os.environ["FOUNDRY_MODEL_NAME"]
11
12 with DefaultAzureCredential() as cred, AIProjectClient(endpoint=endpoint,
13 credential=cred) as project_client:
14     # Preferred access pattern: mint an authenticated OpenAI client and call
15     # the Responses API.
16     with project_client.get_openai_client() as openai_client:
17         response = openai_client.responses.create(
18             model=model,
19             input="Summarize this HR policy in 5 bullet points."
20         )
21     print(response.output_text)

```

3. Foundry Agent Service (Enterprise Runtime & Control Plane)

Foundry Agent Service is the **hosting and operations plane** for enterprise agents.

Responsibilities

- Agent execution and scaling
- Conversation state and memory
- Secure tool execution (built-in tools + MCP)
- Per-agent Entra ID identity
- Telemetry, tracing, governance
- Publishing endpoints (Teams, Copilot, APIs)

Foundry Agent Service **does not define agent logic** — it runs it.

4. Hosted Runtime in Foundry (Correct Meaning)

4.1 Hosted runtime (where code executes)

What “hosted runtime” actually means

A **hosted runtime** means that **the execution environment for an agent runs inside Microsoft Foundry Agent Service infrastructure**, rather than on your VM, container, or on-prem compute.

Key characteristics:

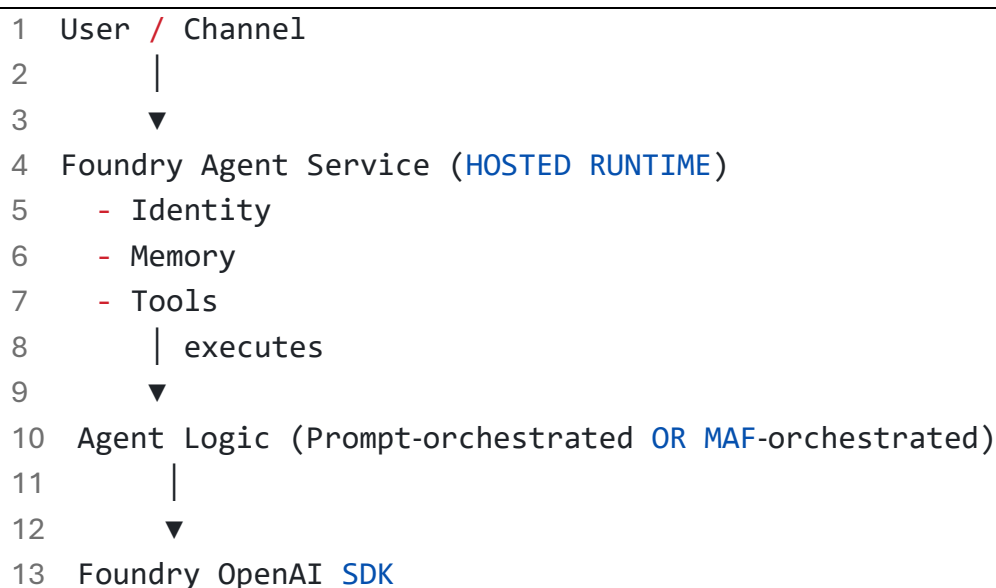
- Foundry owns **scaling, lifecycle, memory, identity, and tool execution sandbox**
- Your logic (prompt-based or MAF-based) executes *within* the Foundry runtime
- You do **not** manage servers, containers, or schedulers

This applies regardless of orchestration style:

- Prompt-orchestrated agents
- Deterministic MAF workflows
- Multi-agent graphs

Important: *Hosted runtime describes **where execution happens** — not how the agent is authored.*

Runtime mental model



4.2 Hosted agent (a specific Foundry resource type)

A **hosted agent** is a **first-class managed resource** in Foundry Agent Service.

It is:

- Created and stored inside Foundry
- Addressable via **agent ID / endpoint**
- Runnable via **threads / runs**
- Publishable to **Teams, Copilot, or REST APIs**

A hosted agent **always uses a hosted runtime**, but **not all hosted runtimes require a hosted agent**.

Hosted agent is NOT:

- NOT equal to “prompt-only agent”
- NOT equal to “no-code agent”
- NOT required for all hosted execution scenarios

4.3 The missing distinction (exam-critical)

Concept	What it describes	Do you need a hosted agent?
Hosted runtime	Where execution runs	✗ Not necessarily
Hosted agent	A managed Foundry agent resource	✓ Only if you want agent endpoints, threads, publishing

This leads to **two distinct hosted patterns** that are often conflated.

4.4 Hosted patterns (correct mapping)

Pattern B1 — Hosted runtime with hosted agent (classic Foundry agent)

- Foundry Agent Service hosts execution
- You **create an agent resource** (agents.create)
- Threads, memory, and tools are managed by Foundry
- Entry point is **agent ID**

Used for:

- Copilots
- Teams / M365 integration
- Conversational assistants

Pattern B2 — Hosted runtime without hosted agent (service-hosted orchestration)

- Execution runs inside Foundry-managed infrastructure
- **No persistent agent resource** is required
- Logic lives in:
 - MAF workflows
 - Application code invoked by Foundry
 - Tool-driven orchestrators

Used for:

- Background workflows
- Case pipelines
- Deterministic processing exposed *behind* an agent or API later

This is where **MAF + Foundry services** often run *without* creating a hosted agent resource.

4.5 Why this distinction matters

Misunderstanding this leads to architectural errors:

- Assuming “hosted” implies prompt-only
- Assuming MAF must be client-hosted
- Over-creating hosted agents where a hosted runtime is sufficient

Rule of thumb:

- If you need **conversation, memory, publishing, or Copilot integration** → create a **hosted agent**
 - If you need **execution, scaling, governance** only → hosted runtime is enough
-

5. Anti-Patterns: Common Wrong Interpretations of “Hosted” (Fix These Fast)

This section is designed to **prevent the most common student mistakes**.

Anti-pattern 1: “Hosted means prompt-only / config-only”

Wrong: Hosted describes *execution location*, not authoring style.

✓ **Correct:** A hosted agent can be prompt-orchestrated **or** MAF-orchestrated.

How to fix in designs: Separate these dimensions:

- **Hosting:** Foundry Agent Service vs your compute
- **Orchestration:** prompt-orchestrated vs MAF workflow

Anti-pattern 2: “Hosted means no-code”

Wrong: You can run **full code** (MAF workflows, custom tools) in a hosted runtime.

✅ **Correct:** “No-code” is a *development approach*, not a runtime property.

Best practice: Teach students to say “**prompt-orchestrated**” instead of “no-code”.

Anti-pattern 3: “If it’s not hosted, it’s just a prototype”

Wrong: Client-hosted systems can be production-grade.

✅ **Correct:** Client-hosted is often used for:

- batch processing
- background jobs
- custom network placement
- specialized compute requirements

Best practice: Make the tradeoff explicit: you own operations (scaling, uptime, security hardening).

Anti-pattern 4: “Hosted automatically means compliant and safe”

Wrong: Hosting does not automatically guarantee correct tool permissions, safe prompts, or grounded answers.

✅ **Correct:** Hosted provides the **control plane** (identity, governance, telemetry). You still must:

- implement permission trimming
- set grounding rules
- define tool scopes
- validate outputs

Best practice: Use evals + policies + observability to prove safety.

Anti-pattern 5: “MAF requires self-hosting”

Wrong: MAF logic can be executed in a hosted runtime.

✅ **Correct:** MAF is an SDK. It can run:

- **client-hosted** (your compute)
 - **Foundry-hosted** (Foundry Agent Service)
-

6. Prompt-Orchestrated Agents (LLM-driven)

These are agents where orchestration happens **implicitly inside the LLM**.

Characteristics

- Behavior primarily defined by instructions
- Tool calling inferred by the model
- Fast to build
- Harder to audit and test deterministically

Best suited for

- RAG Q&A
- Knowledge assistants
- Research helpers

These agents can be **Foundry-hosted** or **client-hosted**, but are most commonly deployed **hosted** for enterprise governance.

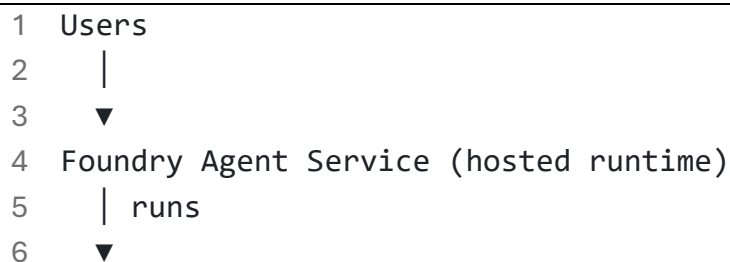
7. Foundry + Microsoft Agent Framework (MAF)

MAF is used when **agent behavior must be explicitly engineered**.

What MAF adds

- Explicit agents and roles
- Graph-based workflows
- Deterministic routing
- Fan-out / fan-in
- Human-in-the-loop approval gates
- Durable checkpoints

Architecture (Hosted execution)



```
7 MAF Agent Graph
8 | calls
9 ▼
10 Foundry OpenAI SDK
```

Minimal Python example – MAF workflow skeleton

```
1 from maf import Agent, Tool, Workflow
2
3 classifier = Agent(
4     name="PolicyClassifier",
5     tools=[Tool.search, Tool.summarize]
6 )
7
8 workflow = Workflow()
9 workflow.add_step(classifier)
```

8. Mapping to Pattern A / B / C (Exam-Critical)

This mapping is based on **two axes**: 1) **Where does the runtime execute?**

(client-hosted vs Foundry-hosted)

2) **How is orchestration implemented?** (prompt-orchestrated vs MAF-orchestrated)

To make this *implementable*, each pattern below includes:

- **Which SDK(s) you use**
- A **minimal code snippet** showing the SDK entry point

Note for students: Names of some MAF classes/functions vary by language/version.

Treat the MAF snippets as **structure + intent**. The Foundry OpenAI SDK examples (via azure.ai.projects) show the concrete Python entry point used in this course.

A — Client-Hosted Orchestration (You host the runtime)

Definition

Your application/service runs orchestration on your compute (VM, container, Functions, on-prem). It calls **Foundry OpenAI SDK** for models. You may still use Foundry resources (projects, connections, eval), but **you are not using Foundry Agent Service as the runtime**.

SDKs used

-  **Foundry OpenAI SDK (Python)** via azure.ai.projects (AIProjectClient.openai...)
-  Optional: **MAF SDK** (if you want deterministic workflow logic)

- **+** Optional: your own connectors (Graph/SharePoint/DB/APIs) depending on the pipeline

Minimal SDK entry point – Foundry OpenAI SDK in a client-hosted service

```

1 # PATTERN A: Your code hosts the runtime and calls Foundry OpenAI SDK
2 from azure.ai.projects import AIProjectClient
3 from azure.identity import DefaultAzureCredential
4
5 project = AIProjectClient.from_connection_string(
6     credential=DefaultAzureCredential(),
7     conn_str="<FOUNDRY_PROJECT_CONNECTION_STRING>"
8 )
9
10 def classify_and_summarize(doc_text: str) -> dict:
11     # In Pattern A, YOU own orchestration (routing, retries, validation, storage)
12     # The only model call is via Foundry OpenAI SDK
13     resp = project.openai.chat.completions.create(
14         model="gpt-4o-mini",
15         messages=[
16             {"role": "system", "content": "Return JSON: {category, summary, confidence}.",},
17             {"role": "user", "content": doc_text}
18         ],
19         response_format={"type": "json_object"} # structured output best practice
20     )
21     return resp.choices[0].message.parsed # pseudo: depending on SDK version, parse JSON yourself

```

Optional Pattern A (deterministic workflow): client-hosted MAF calling Foundry OpenAI SDK

```

1 # PATTERN A + MAF: You still host execution, but you use MAF for workflow structure
2 # (Illustrative – exact import paths depend on the MAF SDK version)
3 from maf import Workflow, Step
4
5 workflow = Workflow(name="nightly_document_pipeline")
6
7 @workflow.step("classify")
8 def step_classify(state):
9     state["classification"] = classify_and_summarize(state["text"]) # calls Foundry OpenAI SDK
10     return state
11
12 @workflow.step("persist")
13 def step_persist(state):
14     # write to DB/data lake
15     return state
16

```

Pattern B — Foundry-Hosted Prompt-Orchestrated Agent

Definition

Foundry Agent Service hosts the runtime and the agent is primarily **prompt-orchestrated** (LLM-driven tool calling and reasoning). This is the fastest path to a governed assistant.

SDKs used

-  **Foundry Agent Service SDK (Python)** via azure.ai.projects agent APIs
-  **Foundry OpenAI SDK** is used *by the agent runtime* for model calls (you typically don't call it directly in Pattern B)
-  Tools configured in Foundry (search/file tools, connectors, MCP tools)

Minimal SDK entry point – Create and run a Foundry-hosted agent (prompt-orchestrated)

```
1  # PATTERN B: Use Foundry Agent Service SDK to create/run a hosted agent
2  from azure.ai.projects import AIProjectClient
3  from azure.identity import DefaultAzureCredential
4
5  project = AIProjectClient.from_connection_string(
6      credential=DefaultAzureCredential(),
7      conn_str="<FOUNDRY_PROJECT_CONNECTION_STRING>"
8  )
9
10 # 1) Create (or reference) an agent configuration in Foundry Agent Service
11 agent = project.agents.create(
12     name="hr-policy-assistant",
13     instructions=(
14         "You answer strictly using the policy corpus. "
15         "If you cannot find support in retrieved documents, say you don't
16         know. "
17         "Always return citations with document IDs."
18     ),
19     tools=[
20         {"type": "file_search"},
21         # Add other tools as allowed (e.g., MCP tools)
22     ],
23     model="gpt-4o-mini" # agent selects model via Foundry configuration
24 )
25
26 # 2) Run a conversation (thread/session) against the hosted agent
27 thread = project.agents.threads.create()
28 project.agents.threads.messages.create(
29     thread_id=thread.id,
30     role="user",
31     content="What is the rule for home office equipment reimbursement?"
32 )
33
34 run = project.agents.threads.runs.create(
35     thread_id=thread.id,
36     agent_id=agent.id
37 )
```

```
38 result = project.agents.threads.runs.get(thread_id=thread.id, run_id=run.id)
39 messages = project.agents.threads.messages.list(thread_id=thread.id)
40 print(messages.data[0].content)
```

What this snippet teaches students

- Pattern B is about using the **Agent Service SDK** (create agent, create thread, run) rather than writing orchestration code.
 - Model access happens through Foundry **behind the agent**, even though the same Foundry OpenAI SDK is the model layer.
-

Pattern C — Foundry-Hosted MAF-Orchestrated System

Definition

Foundry Agent Service hosts the runtime and **MAF provides explicit orchestration** (workflows, multi-agent coordination, approvals).

SDKs used

-  **MAF SDK** (workflow/graph, checkpoints, approvals)
-  **Foundry Agent Service** (hosted runtime, identity, tools, ops) — used to host/expose your orchestrated system as an agent endpoint
-  **Foundry OpenAI SDK** (model calls inside workflow steps)

Minimal SDK entry point – MAF workflow steps calling Foundry OpenAI SDK

```
1 # PATTERN C: MAF defines the workflow; model calls are still via Foundry
  OpenAI SDK
2 from azure.ai.projects import AIProjectClient
3 from azure.identity import DefaultAzureCredential
4
5 project = AIProjectClient.from_connection_string(
6     credential=DefaultAzureCredential(),
7     conn_str="<FOUNDRY_PROJECT_CONNECTION_STRING>"
8 )
9
10 def llm_step(system: str, user: str) -> str:
11     resp = project.openai.chat.completions.create(
12         model="gpt-4o-mini",
13         messages=[
14             {"role": "system", "content": system},
15             {"role": "user", "content": user}
16         ]
17     )
18     return resp.choices[0].message.content
19
20 # (Illustrative) MAF workflow skeleton
```

```

21 from maf import Workflow
22
23 wf = Workflow(name="case_handling")
24
25 @wf.step("intake")
26 def intake(state):
27     state["summary"] = llm_step(
28         "Summarize the case intake text into short bullets.",
29         state["intake_text"]
30     )
31     return state
32
33 @wf.step("classify")
34 def classify(state):
35     state["routing"] = llm_step(
36         "Classify case type (benefit/permit/complaint/other) and provide a
routing code.",
37         state["summary"]
38     )
39     return state
40
41 @wf.step("human_approval")
42 def approval(state):
43     # In real MAF: emit an approval request and pause/checkpoint
44     state["approved"] = False # placeholder
45     return state

```

(Illustrative) Hosting the MAF system behind Foundry Agent Service

There are two common ways to connect MAF to Foundry hosting:

1) **Foundry hosts the agent endpoint** and invokes your orchestrator logic (MAF) internally. 2) **Foundry hosts an agent** that calls your MAF orchestrator as a secure tool/API.

Below is a minimal sketch of option (2), which is easy to explain and widely applicable:

```

1 # PATTERN C (common integration): Foundry-hosted agent + a tool that calls a
MAF orchestrator API
2 # You deploy your MAF workflow behind an internal API (Function/App
Service/K8s)
3 # Then register that API as a tool available to the Foundry-hosted agent.
4
5 agent = project.agents.create(
6     name="case-handling-agent",
7     instructions="You manage cases by calling the CaseWorkflow tool and then
requesting approvals.",
8     tools=[
9         {
10             "type": "function",

```

```

11         "name": "CaseWorkflow",
12         "description": "Runs the MAF case workflow and returns next
action + state.",
13         "parameters": {
14             "type": "object",
15             "properties": {
16                 "case_id": {"type": "string"},
17                 "intake_text": {"type": "string"}
18             },
19             "required": ["case_id", "intake_text"]
20         }
21     },
22 ],
23     model="gpt-4o-mini"
24 )

```

What this teaches students

- Pattern C uses **MAF** to make the process deterministic and auditable.
 - Model calls inside steps still use the **Foundry OpenAI SDK**.
 - Foundry Agent Service provides the **hosted runtime + identity + tools** layer around the orchestrated system.
-

9. Use Cases by Pattern + Best Practices (What to Use When)

Pattern A use cases (Client-Hosted)

Typical use cases

- Document processing pipelines (classify → extract → validate → store)
- Scheduled reporting and summarization
- Agentic batch jobs that must run near data or inside restricted networks

Best practices

- Add structured output contracts (JSON schema) + validation
 - Implement retries/timeouts and idempotency
 - Centralize telemetry (trace IDs across tools)
 - Use strict permission trimming when accessing enterprise content
-

Pattern B use cases (Foundry-Hosted Prompt-Orchestrated)

Typical use cases

- HR/policy copilot (Q&A + summarization)
- Internal knowledge base assistant

- “Ask my SharePoint” experiences

Best practices

- Ground answers to retrieved content (citations, quotes, doc IDs)
 - Use a scope classifier (“in scope / out of scope”) to avoid hallucinations
 - Limit tool permissions (least privilege) and log tool invocations
 - Evaluate regularly (hallucination checks, refusal quality, retrieval quality)
-

Pattern C use cases (Foundry-Hosted MAF)

Typical use cases

- Public sector case flows (intake → classify → verify → decide → archive)
- HR lifecycle automation (hire, change, terminate) with approvals
- Compliance investigations with evidence tracking

Best practices

- Make workflow state explicit (case IDs, checkpoints)
 - Gate high-risk actions with human approvals
 - Use deterministic routing rules where possible
 - Use prompts only for bounded reasoning steps, never for process control
-

10. Concrete Mini Architecture Examples (Pattern-Based)

This section provides **concrete, end-to-end mini architectures** that students can reason about, sketch on a whiteboard, or implement as labs. Each example maps **directly** to one of the core patterns.

Example A — RAG Policy Assistant (Pattern B: Foundry-Hosted, Prompt-Orchestrated)

Scenario

Employees ask questions about HR, security, or compliance policies stored in SharePoint and PDFs. The system must:

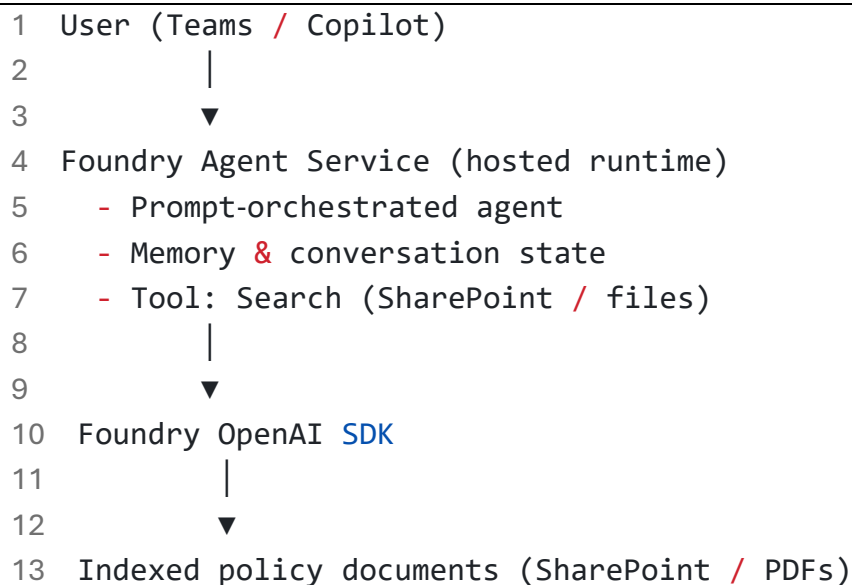
- Answer only using approved documents
- Avoid hallucinations
- Be fast to build and governed

Chosen pattern: Pattern B (Foundry-hosted prompt-orchestrated agent)

Why this pattern fits

- No deterministic workflow required
- Low branching
- Knowledge-centric task
- Strong value from Foundry governance and memory

Mini architecture



Key implementation ideas (best practice)

- Add a **scope classifier**: “Is this question about policy?”
 - Force answers to include **citations or document IDs**
 - Refuse to answer when retrieval returns no results
 - Periodically run hallucination and grounding evals
-

Example B — Case-Handling Workflow (Pattern C: Foundry-Hosted, MAF-Orchestrated)

Scenario

A public-sector case-handling process (e.g. benefits, permits, complaints) with:

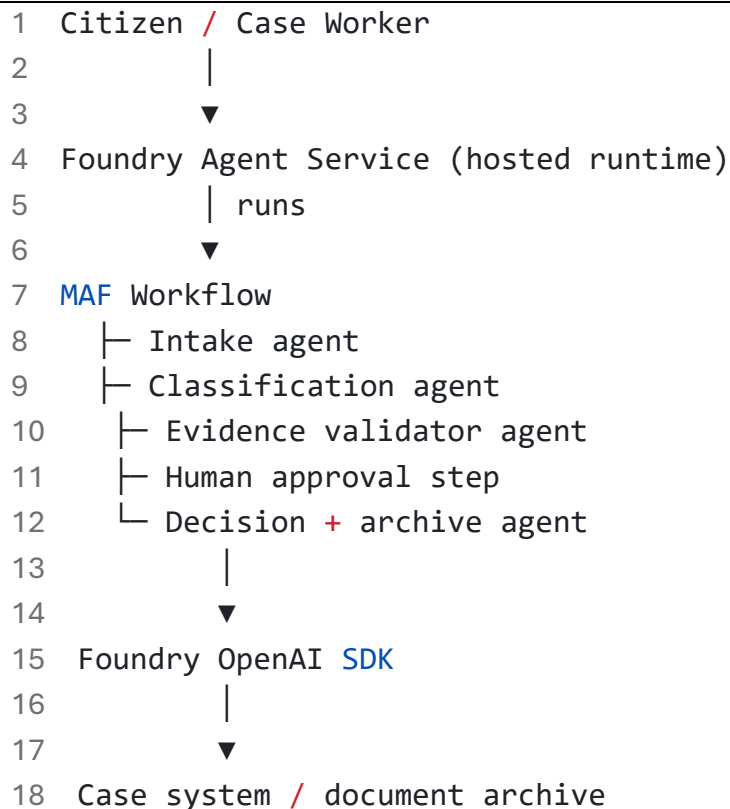
- Multiple steps
- Human approvals
- Full auditability
- Ability to pause and resume

Chosen pattern: Pattern C (Foundry-hosted + MAF)

Why this pattern fits

- Human approvals required
- Deterministic routing
- Legal and audit requirements
- Long-running stateful process

Mini architecture



Key implementation ideas (best practice)

- Every step writes to a **case state object** (case ID, status, decisions)
- Use **deterministic routing rules** where possible
- Gate decisions with explicit **human-in-the-loop approvals**
- Keep prompts focused on *analysis*, not *process control*

Example C — Batch Document Pipeline (Pattern A: Client-Hosted Runtime)

Scenario

A nightly job processes thousands of documents to:

- Classify content
- Extract fields
- Summarize

- Store structured results

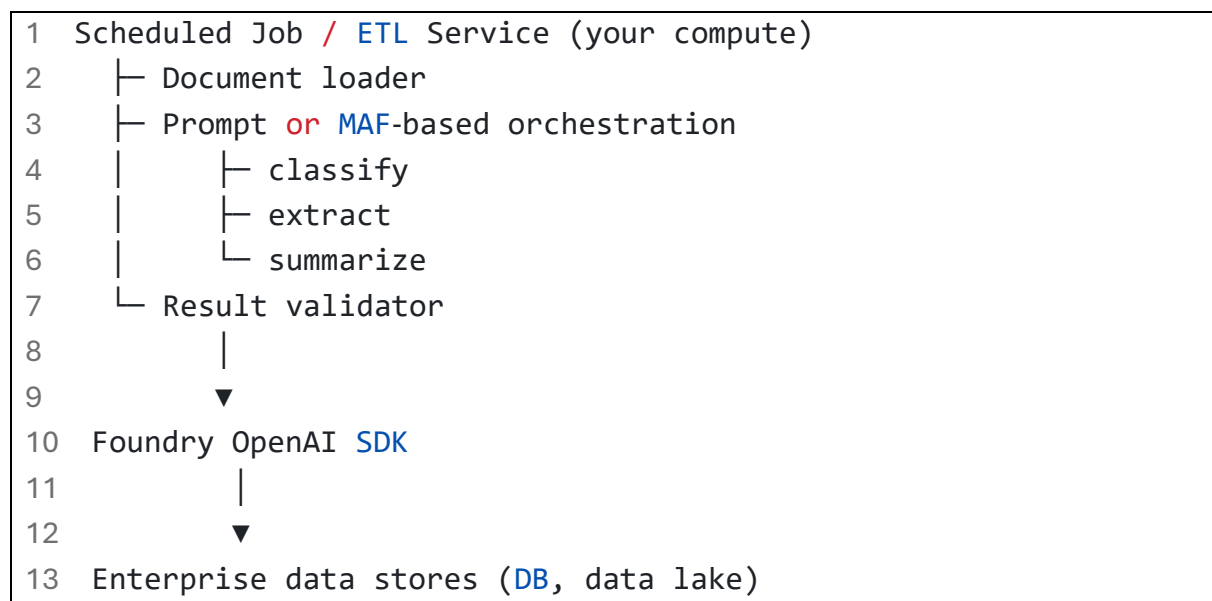
This pipeline must run **near the data**, possibly on-prem or in a private network.

Chosen pattern: Pattern A (client-hosted runtime)

Why this pattern fits

- No interactive users
- Batch / scheduled execution
- Tight control of networking and compute
- Pipeline-style processing

Mini architecture



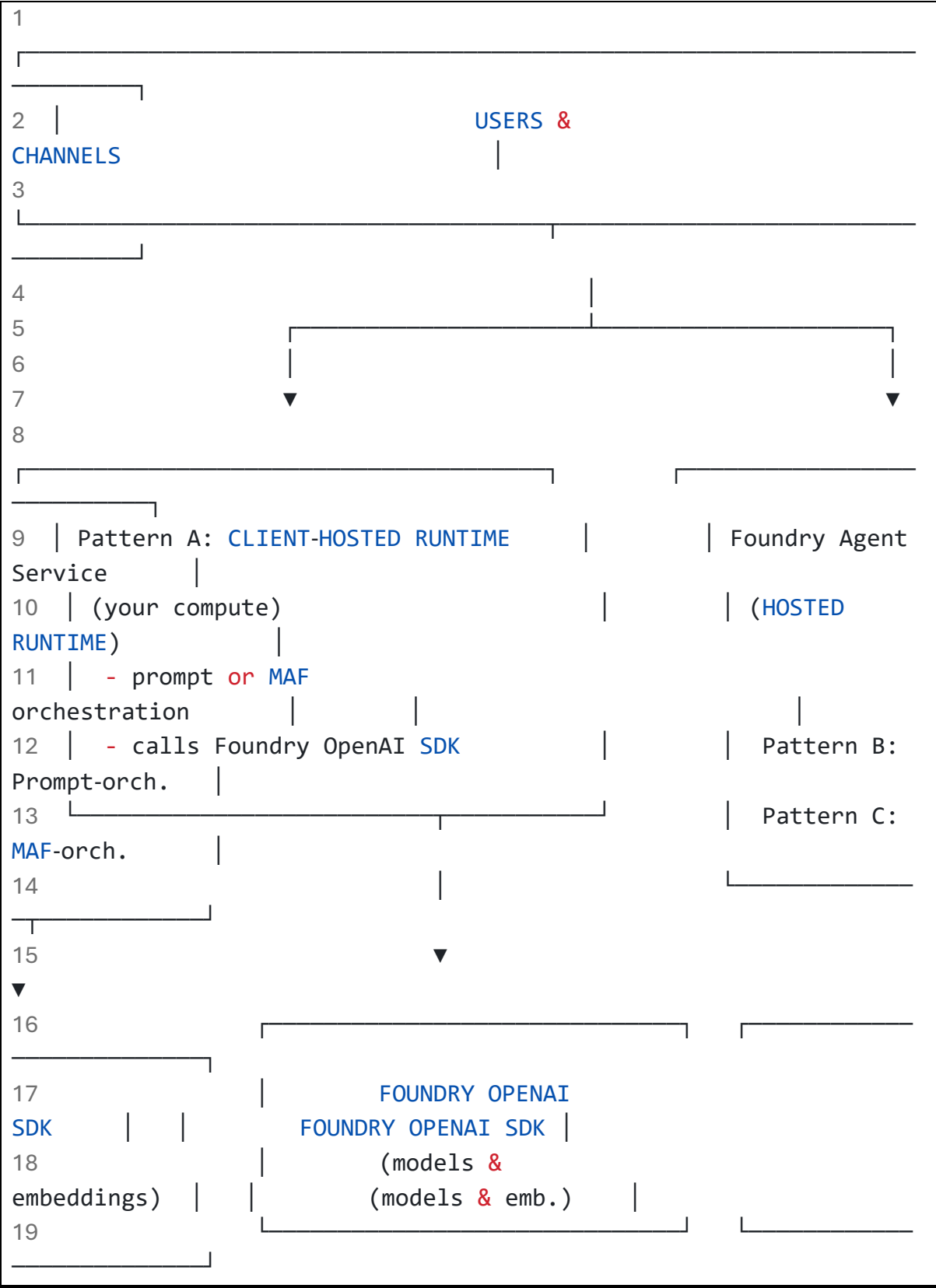
Key implementation ideas (best practice)

- Enforce **structured outputs** (JSON schema) at every step
- Validate outputs before persistence
- Implement retries and idempotency
- Centralize logs and trace IDs across batch steps

Pattern Comparison Summary (Teaching Aid)

Example	Pattern	Hosting	Orchestration	Core strength
RAG Policy Assistant	B	Foundry	Prompt-based	Speed + governance
Case-Handling Workflow	C	Foundry	MAF	Determinism + audit
Batch Pipeline	A	Client	Prompt or MAF	Control + scalability

11. Visual Architecture Overview (with Patterns)



12. Final Rule of Thumb

- **Hosted** describes *where agents run*
- **Prompt vs MAF** describes *how they reason*

If correctness, approvals, resumability, or auditability matter → **Pattern C**.

If you need a governed assistant fast → **Pattern B**.

If you must control runtime placement or build pipelines → **Pattern A**.
